

XÂY DỰNG HỆ THỐNG SELF-EVOLVING AI AGENTS TRONG XỬ LÝ SỰ CỐ MẠNG VIỄN THÔNG

Nguyễn Thanh Hải

optimus1072005@gmail.com

Nguyễn Duy Hưng¹

hungnd1235@gmail.com

Nguyễn Duy Anh¹

nguyenduyanh821998@gmail.com

Bùi Quang Hưng¹

contact.hung.bq@gmail.com

¹Mentor, Trung tâm Chuyển đổi số, Tổng công ty Mạng lưới Viettel

Lưu ý. Báo cáo được trình bày súc tích theo template đề tài nghiên cứu dành cho sinh viên báo cáo với mentor, tập trung làm nổi bật đóng góp trực tiếp của sinh viên, tính mới, tính sáng tạo và hiệu quả thực nghiệm. Tổng nội dung báo cáo được kiểm soát không vượt quá 6 trang.

1. GIỚI THIỆU CHUNG

Các mạng viễn thông hiện đại, đặc biệt trong bối cảnh 5G, trung tâm dữ liệu và mạng điều khiển bằng phần mềm, có quy mô lớn, nhiều lớp giao thức và phụ thuộc chặt chẽ giữa thiết bị, dịch vụ, cấu hình định tuyến, topology và chỉ số hiệu năng. Một sự cố nhỏ như sai ASN BGP, thiếu địa chỉ IP, link down, switch BMv2 dừng hoạt động hoặc rule SDN sai có thể gây mất kết nối diện rộng, suy giảm chất lượng dịch vụ và làm tăng chi phí vận hành. Trong thực tế, xử lý sự cố không chỉ là phát hiện bất thường, mà còn cần định vị thiết bị lỗi, xác định nguyên nhân gốc rễ (Root Cause Analysis – RCA) và đưa ra hướng khắc phục có thể kiểm chứng. Các khảo sát về RCA tự động cho thấy bài toán này khó vì triệu chứng quan sát được thường gián tiếp, chịu nhiễu, và có thể bắt nguồn từ nhiều tầng cấu hình khác nhau [6].

Các cách tiếp cận truyền thống thường dựa trên luật thủ công, kinh nghiệm chuyên gia hoặc mô hình học máy tĩnh. Những phương pháp này khó thích ứng khi topology thay đổi, khi xuất hiện dạng lỗi mới hoặc khi dữ liệu vận hành có nhiễu. Sự phát triển của mô hình ngôn ngữ lớn (LLM) và AI Agent mở ra khả năng mới: agent có thể đọc mô tả sự cố, sử dụng công cụ chẩn đoán, suy luận nhiều bước và giải thích kết quả bằng ngôn ngữ tự nhiên. Các nghiên cứu gần đây về LLM cho chẩn đoán mạng nhấn mạnh tiềm năng kết hợp tri thức miền, log vận hành và công cụ kiểm chứng thay vì chỉ dựa vào câu trả lời một lượt của mô hình [14].

Tuy nhiên, agent thông thường vẫn mang tính tĩnh; nếu prompt, bộ nhớ hoặc mô tả công cụ không được cập nhật thì hiệu quả chẩn đoán dễ suy giảm khi môi trường mạng thay đổi.

Đề tài này nghiên cứu và xây dựng hệ thống **Self-Evolving AI Agents** cho xử lý sự cố mạng viễn thông trên nền tảng benchmark NIKA [8]. Khác với agent tĩnh, self-evolving agent khai thác chính lịch sử xử lý nhiệm vụ, phản hồi và kết quả đánh giá làm nguồn dữ liệu để cập nhật bộ nhớ, công cụ hoặc chiến lược suy luận [5]. Mục tiêu của đề tài là biến mỗi lần chẩn đoán thành một nguồn kinh nghiệm giúp agent cải thiện ở các lần sau. Cụ thể, hệ thống cần tiếp nhận dữ liệu từ môi trường mạng giả lập, gọi các công cụ kiểm tra trạng thái mạng, suy luận nguyên nhân gốc rễ, gửi kết quả đánh giá và cập nhật tri thức chẩn đoán dài hạn.

Ý nghĩa thực tiễn của sản phẩm nằm ở việc giảm tải cho kỹ sư vận hành trong các bước lặp lại: kiểm tra kết nối, đọc cấu hình host/router, đối chiếu bảng định tuyến, phân tích log switch và tổng hợp bằng chứng. Trong bối cảnh vận hành thực tế, một trợ lý như vậy không thay thế chuyên gia, nhưng có thể đóng vai trò “first responder” để thu hẹp không gian giả thuyết, chuẩn hóa báo cáo sự cố và lưu lại kinh nghiệm đã được kiểm chứng cho các ca tương tự.

Đóng góp chính của sinh viên gồm:

- Thiết kế quy trình agent chẩn đoán trên nền tảng NIKA/Kathará, tích hợp các dịch vụ MCP để agent có thể tương tác với router, host, switch P4/BMv2, telemetry và hệ thống chấm điểm.
- Đề xuất và hiện thực mô-đun **Procedural Memory**, lấy cảm hứng từ Skill-Pro, lưu trữ kỹ năng chẩn đoán dạng thủ tục và tái sử dụng kinh nghiệm phù hợp với ngữ cảnh sự cố.
- Đề xuất và hiện thực mô-đun **Tool Refinement**, lấy

cảm hứng từ DRAFT, cho phép agent tự tinh chỉnh tài liệu mô tả công cụ MCP dựa trên các lần gọi công cụ thực tế mà không thay đổi mã nguồn công cụ.

- Thực nghiệm so sánh agent baseline và agent tự tiến hóa trên benchmark NIKA, đánh giá theo detection, localization, RCA, số bước, số lần gọi công cụ và chi phí token.

2. NỘI DUNG VÀ PHƯƠNG PHÁP

2.1. Quy trình triển khai

Đề tài được triển khai theo quy trình gồm bốn lớp. Lớp thứ nhất là môi trường mạng giả lập dựa trên Kathará [7], trong đó mỗi router, host hoặc switch được chạy trong container và có thể được tiêm lỗi có kiểm soát. Lớp thứ hai là hệ thống MCP server [1], chuẩn hóa các thao tác chẩn đoán thành công cụ để agent gọi trong quá trình suy luận. Lớp thứ ba là workflow agent xây dựng bằng LangGraph [3], hỗ trợ các luồng ReAct [13], Plan & Execute và Reflexion [11]. Lớp cuối cùng là hai mô-đun tự tiến hóa: Procedural Memory và Tool Refinement.

Một episode chẩn đoán diễn ra theo trình tự: khởi động lab mạng, tiêm lỗi, chạy agent, thu thập trace gọi công cụ, nộp kết quả gồm `is_anomaly`, `faulty_devices`, `root_cause_name`, sau đó đánh giá và kích hoạt học ngoại tuyến. Với baseline, mỗi episode độc lập. Với evolved agent, bộ nhớ kỹ năng và tài liệu công cụ đã được tinh chỉnh từ các episode trước được nạp lại vào ngữ cảnh để hỗ trợ case tiếp theo.

Luồng xử lý được tách thành hai pha có chủ đích. Trong *Diagnosis Phase*, agent được quyền dùng đầy đủ công cụ để thu thập bằng chứng, kiểm tra giả thuyết và loại trừ nguyên nhân sai. Trong *Submission Phase*, agent chỉ còn công cụ nộp kết quả, nhờ đó giảm rủi ro vừa chẩn đoán vừa thay đổi kết luận cuối cùng một cách thiếu kiểm soát. Thiết kế này bám sát tinh thần ReAct, nơi suy luận và hành động công cụ được xen kẽ để giải quyết tác vụ nhiều bước [13], nhưng bổ sung lớp đánh giá sau episode để phục vụ học liên tục.

2.2. Công cụ và công nghệ sử dụng

Hệ thống sử dụng NIKA làm benchmark chẩn đoán mạng, Kathará cho ảo hóa topology, FastMCP cho server công cụ, LangGraph/LangChain cho điều phối agent, InfluxDB cho telemetry và Python 3.12 với uv để quản lý môi trường. Với các kịch bản định tuyến, hệ thống tương tác với FRRouting để đọc cấu hình và trạng thái route [4]; với kịch bản data-plane lập trình được, hệ thống đọc log và bảng hành vi của P4/BMv2, dựa trên mô hình lập trình packet processor của P4 [2]. Các MCP server chính gồm:

- **Kathará Base**: kiểm tra reachability, ping, iperf, cấu hình host, netstat và thực thi shell trong container.
- **FRR**: đọc cấu hình OSPF/BGP, bảng định tuyến và thực thi lệnh `vttysh`.
- **BMv2/P4**: đọc log switch, dump bảng luồng, counter và register.
- **Telemetry**: truy vấn InfluxDB để lấy chỉ số mạng theo thời gian.
- **Task Management**: nhận kết quả cuối cùng để chấm điểm tự động.
- **Tool Evolution**: cung cấp tài liệu công cụ đã được tinh chỉnh cho agent.

2.3. Mô-đun Procedural Memory

Procedural Memory được thiết kế để lưu trữ kinh nghiệm chẩn đoán lâu dài dưới dạng `ProceduralSkill`, lấy cảm hứng từ hướng procedural memory trong Skill-Pro [9]. Mỗi kỹ năng gồm điều kiện kích hoạt, chuỗi bước thực thi, điều kiện kết thúc, thuộc tính ngữ cảnh như scenario, protocol, service, symptom, tool và thống kê sử dụng như maturity, success count, failure count. Trạng thái bộ nhớ được lưu tại `runtime/memory/{bank_id}/skills.json`.

Điểm mới của mô-đun là biểu diễn kinh nghiệm thành các quy trình chẩn đoán ngắn gọn, có điều kiện kích hoạt và có thể tái sử dụng. Khi bắt đầu một incident, hệ thống truy xuất kỹ năng phù hợp theo ngữ cảnh như scenario, protocol, service, symptom và tool. Điều này giúp tránh áp dụng sai kinh nghiệm, ví dụ không dùng quy trình kiểm tra BGP cho một case P4/BMv2 hoặc không dùng quy trình kiểm tra DHCP cho lỗi SDN controller.

Sau mỗi episode, hệ thống tính phần thưởng kết hợp từ detection, localization và RCA; lưu episode vào experience pool; sinh Semantic Gradient để rút kinh nghiệm; tạo nhiều ứng viên kỹ năng mới; rồi dùng PPO Gate phi tham số [10] để chỉ chấp nhận kỹ năng có cải thiện đủ rõ so với baseline. Điều kiện thành công nghiêm ngặt giúp giảm nguy cơ học vẹt đáp án benchmark hoặc ghi nhớ tri thức sai.

Cơ chế học được kiểm soát theo ba nguyên tắc. Thứ nhất, chỉ những episode đạt ngưỡng chất lượng mới được đưa vào golden pool, tránh để kết luận sai trở thành tri thức lâu dài. Thứ hai, kỹ năng mới phải vượt qua PPO Gate, tức là có lợi ích tương đối so với kỹ năng hiện có thay vì chỉ diễn đạt lại cùng một quy trình. Thứ ba, thư viện kỹ năng được giới hạn kích thước và có cơ chế nghỉ hưu kỹ năng kém hiệu quả, giúp bộ nhớ không phình to và không gây nhiễu ngữ cảnh cho LLM.

2.4. Mô-đun Tool Refinement

Tool Refinement tập trung vào việc giúp agent hiểu và sử dụng công cụ MCP tốt hơn, lấy cảm hứng từ DRAFT [12]. Thay vì sinh công cụ mới hoặc sửa mã nguồn công cụ, hệ thống tự cải thiện phần mô tả công cụ dựa trên dữ liệu thực thi. Mỗi tài liệu công cụ gồm description, preconditions, hướng dẫn tham số, failure modes, usage notes, ví dụ đúng/sai, mastery score và convergence score. Trạng thái được lưu tại `runtime/tool_evolution/{library_id}/state.json`.

Quy trình học gồm ba bước. Explorer trích xuất các lần gọi công cụ từ `messages.jsonl`. Analyzer phát hiện khoảng trống hiểu biết như sai schema tham số, dùng tên thiết bị không tồn tại hoặc không biết diễn giải đầu ra. Rewriter viết lại tài liệu công cụ để bổ sung ràng buộc, ví dụ và hướng dẫn chẩn đoán. Khi mastery và convergence đạt ngưỡng, tài liệu được đóng băng adaptive để tránh viết lại không cần thiết.

Đề tài chọn cách tinh chỉnh tài liệu công cụ vì phù hợp với yêu cầu an toàn trong môi trường mạng. Công cụ nguyên thủy không bị thay đổi mã nguồn; agent chỉ học cách gọi đúng tham số, hiểu đúng tiền điều kiện và diễn giải đúng kết quả trả về. Điều này giảm rủi ro tạo ra thao tác không mong muốn trên thiết bị mạng, đồng thời vẫn cải thiện khả năng sử dụng tool theo thời gian.

3. KẾT QUẢ THỰC HIỆN

3.1. Sản phẩm đạt được

Sản phẩm của đề tài là một hệ thống agent chẩn đoán sự cố mạng có khả năng tự cải thiện qua nhiều episode. Hệ thống không chỉ chạy một agent LLM gọi công cụ, mà còn bổ sung lớp học kinh nghiệm sau khi agent hoàn thành nhiệm vụ. Các thành phần đã hoàn thiện gồm:

- Workflow chẩn đoán ReAct tích hợp MCP tools, có pha diagnosis và pha submission tách biệt để hạn chế agent gửi kết quả khi chưa đủ bằng chứng.
- Procedural Memory Store, SkillToolRuntime và cơ chế tiến hóa kỹ năng sau episode, cho phép kỹ năng chẩn đoán được truy xuất, kích hoạt, cập nhật và nghỉ hưu theo chất lượng sử dụng.
- Tool Refinement Store, ToolEvolutionRuntime và pipeline Explorer–Analyzer–Rewriter, cho phép tài liệu công cụ tự cải thiện dựa trên trace gọi công cụ thực tế.
- Bộ script benchmark tuần tự để chạy baseline và evolved agent trên cùng tập incident, lưu log, tính metric và so sánh hiệu quả.

Ở mức triển khai, sản phẩm có ba đặc điểm quan trọng. Thứ nhất, trạng thái học được lưu ngoài phiên

chạy nên có thể tái sử dụng giữa các incident thay vì chỉ tồn tại trong context tạm thời của LLM. Thứ hai, mọi cập nhật đều dựa trên trace có thể kiểm tra lại, gồm lời gọi công cụ, tham số, kết quả trả về và điểm đánh giá cuối. Thứ ba, các mô-đun tiến hóa được thiết kế như phần mở rộng quanh agent hiện có, do đó có thể bật/tắt để so sánh công bằng với baseline và để phân tích chi phí của từng cơ chế.

3.2. Benchmark và phạm vi thử nghiệm

Hai lần chạy thực nghiệm đầy đủ tương ứng với baseline và evolved agent, mỗi lần có 125 case được ghi nhận đầy đủ metric. Các case này bao phủ nhiều biến thể topology và 6 nhóm nguyên nhân gốc rễ trên 8 kịch bản mạng. Ngoài các sự cố có lỗi, benchmark còn có các case `no_fault` để kiểm tra khả năng không báo động sai của agent.

Bộ benchmark có ý nghĩa vì nó buộc agent phải kết hợp nhiều nguồn bằng chứng. Một case BGP có thể yêu cầu kiểm tra neighbor, ASN, route advertisement và reachability; một case host có thể cần đối chiếu IP, gateway, DNS và subnet; còn một case P4/BMv2 có thể cần đọc bảng luồng hoặc counter thay vì chỉ ping. Nhờ vậy, kết quả không chỉ phản ánh khả năng trả lời ngôn ngữ tự nhiên, mà còn phản ánh năng lực lập kế hoạch sử dụng công cụ và RCA trong môi trường mô phỏng gần với vận hành mạng.

3.3. Tính năng nổi bật và điểm mới

Điểm mới quan trọng nhất là hệ thống tiến hóa ở hai tầng hỗ trợ nhau. Ở tầng chiến lược, Procedural Memory giúp agent nhớ quy trình chẩn đoán có điều kiện kích hoạt rõ ràng, ví dụ ưu tiên kiểm tra cấu hình IP/gateway khi gặp host mất kết nối hoặc kiểm tra BGP neighbor và route table khi có dấu hiệu route leak. Ở tầng thao tác, Tool Refinement giúp agent hiểu cách gọi công cụ chính xác hơn, đặc biệt với các công cụ có ràng buộc tham số hoặc đầu ra khó diễn giải.

Sinh viên trực tiếp hiện thực các module lưu trữ trạng thái JSON, cơ chế truy xuất kỹ năng, runtime gắn hướng dẫn kỹ năng vào quá trình gọi tool, pipeline tinh chỉnh tài liệu công cụ và quy trình benchmark/evaluation. Các kết quả này tạo thành một prototype hoàn chỉnh có thể chạy lặp qua nhiều incident và tích lũy tri thức thay vì chỉ xử lý từng case rời rạc.

Một điểm đáng chú ý là các cải tiến không yêu cầu huấn luyện lại mô hình nền. Toàn bộ quá trình học diễn ra ở lớp hệ thống: tổ chức lại tri thức thủ tục, tăng chất lượng mô tả công cụ và chọn lọc kinh nghiệm dựa trên reward. Cách làm này phù hợp với môi trường nghiên cứu có tài nguyên hạn chế, đồng thời dễ kiểm soát hơn so với fine-tuning trực tiếp một LLM lớn. Khi cần triển khai thử nghiệm với mô hình khác, các file `skills.json`

Bảng 1. Tóm tắt phạm vi benchmark trong hai lần chạy

Nhóm sự cố	Ví dụ	Số lượng
Link Failures	<code>link_down</code> , <code>link_flap</code> , packet corruption	12
End-Host Failures	thiếu IP, sai gateway, sai DNS, IP conflict	16
Misconfigurations	sai ASN BGP, thiếu route, sai OSPF area, flow rule loop, P4 table misconfig	27
Resource Contention	quá tải sender/receiver, incast, load balancer overload	13
Network Node Errors	FRR down, BMv2 switch down, SDN controller crash, MPLS label limit	20
Network Under Attack	BGP hijacking, DHCP spoofing, ARP poisoning, ACL block, web DoS	12
<code>no_fault</code>	mạng bình thường, không tiêm lỗi	25

và `state.json` vẫn có thể được tái sử dụng như tri thức hệ thống.

3.4. Minh chứng từ hệ thống chạy thực tế

Trong quá trình benchmark, trace của agent cho thấy lỗi phổ biến của baseline không chỉ nằm ở suy luận cuối cùng mà còn ở cách thu thập bằng chứng. Baseline đôi khi gọi công cụ đúng loại nhưng sai thiết bị, kiểm tra reachability trước khi xác định topology liên quan, hoặc đọc kết quả route table nhưng không liên hệ với nguyên nhân gốc rễ. Sau khi Tool Refinement cập nhật tài liệu công cụ, các ghi chú về tiền điều kiện, tham số hợp lệ và cách diễn giải lỗi giúp giảm các lần gọi công cụ kém giá trị. Sau khi Procedural Memory tích lũy kinh nghiệm, agent có xu hướng đi theo quy trình ổn định hơn: xác nhận triệu chứng, khoanh vùng thiết bị, kiểm tra cấu hình liên quan, rồi mới kết luận RCA.

Ghi chú. Mã nguồn, cấu hình chạy và kết quả thực nghiệm được đính kèm trong repository dự án. Các thành phần chính nằm trong `src/nika`, dữ liệu học của Procedural Memory nằm tại `runtime/memory`, còn trạng thái Tool Refinement nằm tại `runtime/tool_evolution`.

4. ĐÁNH GIÁ HIỆU QUẢ

4.1. Tiêu chí đánh giá

Hiệu quả của hệ thống được đánh giá theo hai nhóm tiêu chí. Nhóm thứ nhất là độ chính xác chẩn đoán: detection score đo khả năng phát hiện có lỗi hay không; localization accuracy/F1 đo khả năng xác định đúng thiết bị lỗi; RCA accuracy/F1 đo khả năng xác định đúng nguyên nhân gốc rễ; incident success rate yêu cầu đúng đồng thời cả detection, localization và RCA. Nhóm thứ hai là hiệu năng vận hành: số bước suy luận, số lần gọi công cụ và lượng token tiêu thụ trung bình mỗi incident.

Ba tăng metric này được chọn vì phản ánh quy trình xử lý sự cố trong thực tế. Detection đúng nhưng localization sai vẫn chưa đủ để kỹ sư xử lý sự cố; localization đúng nhưng RCA sai có thể dẫn đến thao tác khắc phục không hiệu quả. Vì vậy, incident

success rate là tiêu chí khắt khe nhất, còn F1 ở localization/RCA giúp đánh giá các trường hợp agent tìm đúng một phần tập thiết bị hoặc nhân nguyên nhân. Việc đo thêm steps, tool calls và tokens giúp phân biệt giữa cải thiện do suy luận tốt hơn và cải thiện phải trả bằng chi phí vận hành quá lớn.

Hai cấu hình được so sánh trên cùng tập incident:

- **Baseline Agent:** ReAct agent tiêu chuẩn, không dùng memory và không dùng tool evolution.
- **Evolved Agent:** ReAct agent tích hợp Procedural Memory và Tool Refinement, có khả năng học liên tục qua các episode.

4.2. Kết quả so sánh

Kết quả cho thấy agent tự tiến hóa cải thiện rõ nhất ở RCA và incident success rate. RCA F1 tăng từ 0.304 lên 0.387, còn số incident giải quyết đúng hoàn toàn tăng từ 26/125 lên 35/125. Đây là chỉ dấu quan trọng vì mục tiêu của xử lý sự cố mạng không chỉ là phát hiện bất thường, mà còn phải chỉ ra đúng thiết bị và nguyên nhân để kỹ sư có thể hành động.

Về hiệu năng, Evolved Agent giảm số bước trung bình 16.7% và số lần gọi công cụ 18.5%. Tool error rate cũng giảm từ 11/1700 tool calls xuống 0/1386 tool calls, cho thấy Tool Refinement giúp agent gọi công cụ ổn định hơn và ít sai tham số hơn. Đối lại, tổng token tăng từ 16.641M lên 18.699M (+12.4%), chủ yếu do input tokens tăng khi hệ thống nạp thêm kỹ năng và tài liệu công cụ đã tinh chỉnh vào prompt. Tổng thời gian chạy tăng từ 207.7 lên 222.6 phút (+7.2%), vẫn nhỏ hơn mức tăng token nhờ số bước và số tool calls giảm.

Nếu xét theo giá trị vận hành, mức tăng 34.6% ở incident success rate là kết quả quan trọng nhất. Trong một hệ thống hỗ trợ kỹ sư, mỗi incident được giải quyết đúng hoàn toàn có thể giảm thời gian đọc log thủ công, giảm số hướng điều tra sai và tạo ra một bản ghi kinh nghiệm cho các lần sau. Mức giảm tool calls cũng có ý nghĩa vì nhiều công cụ chẩn đoán mạng có thể tốn thời gian hoặc tạo tải phụ lên môi trường kiểm thử. Điểm cần cân nhắc là token tăng, do đó khi mở rộng lên hệ

Bảng 2. So sánh baseline và evolved agent trên 125 case chung

Chỉ số	Baseline	Evolved	Thay đổi
Detection score	0.840	0.872	+3.8%
Localization accuracy	0.368	0.384	+4.3%
Localization F1	0.411	0.459	+11.8%
RCA accuracy	0.304	0.376	+23.7%
RCA F1	0.304	0.387	+27.2%
Incident success rate	26/125 (20.8%)	35/125 (28.0%)	+34.6%
Steps trung bình	15.29	12.74	-16.7%
Tool calls trung bình	13.60	11.09	-18.5%
Tool error rate	11/1700 (0.65%)	0/1386 (0.00%)	-100.0%
Input tokens (tổng)	15.944M	17.992M	+12.9%
Output tokens (tổng)	0.697M	0.708M	+1.5%
Total tokens (tổng)	16.641M	18.699M	+12.4%
Tổng thời gian chạy	207.7 phút	222.6 phút	+7.2%

thống thật cần cơ chế nén kỹ năng, chọn top-k tài liệu công cụ và giới hạn context theo loại sự cố.

4.3. Phân tích theo nhóm lỗi

Evolved Agent cải thiện mạnh nhất ở nhóm lỗi host như `host_missing_ip`, `host_incorrect_ip`, `host_incorrect_gateway`. Các kỹ năng tích lũy từ case trước giúp agent nhanh chóng kiểm tra cấu hình host, subnet, gateway và reachability thay vì thử nhiều hướng không liên quan. Với các lỗi định tuyến BGP/OSPF phức tạp, hệ thống có cải thiện nhưng vẫn chưa ổn định vì cần suy luận đồng thời trên neighbor state, route advertisement và quan hệ topology. Nhóm resource contention và một số lỗi tấn công vẫn khó do tín hiệu quan sát không trực tiếp, cần thêm telemetry hoặc cơ chế kiểm chứng giả thuyết mạnh hơn.

Nhìn chung, hai mô-đun tiến hóa tạo ra hiệu quả hỗ trợ: Procedural Memory cải thiện chiến lược chẩn đoán, Tool Refinement cải thiện cách dùng công cụ. Đây là điểm khác biệt so với agent tĩnh chỉ dựa vào prompt ban đầu và lịch sử hội thoại ngắn hạn.

4.4. Hạn chế và rủi ro

Hệ thống vẫn có một số hạn chế. Thứ nhất, hiệu quả tự tiến hóa phụ thuộc vào chất lượng episode trước đó; nếu feedback hoặc ground truth không đáng tin cậy, bộ nhớ có thể ghi nhận quy trình sai. Thứ hai, các kỹ năng học từ một nhóm topology có thể quá khớp với tên thiết bị, giao thức hoặc cách tổ chức lab cũ, làm giảm khả năng chuyển sang topology mới. Thứ ba, Tool Refinement cải thiện mô tả công cụ nhưng không làm tăng năng lực quan sát nếu môi trường không cung cấp telemetry hoặc log cần thiết. Cuối cùng, việc đưa thêm kỹ năng và tài liệu vào prompt làm tăng token, nên cân cơ chế chọn lọc ngữ cảnh chặt chẽ hơn khi triển khai ở

quy mô lớn.

Các rủi ro này cho thấy self-evolving agent cần được vận hành theo nguyên tắc có kiểm soát. Trong môi trường mạng quan trọng, hệ thống nên ghi rõ bằng chứng cho từng kết luận, cho phép kỹ sư phê duyệt kỹ năng mới trước khi lưu lâu dài và có cơ chế rollback bộ nhớ khi phát hiện tri thức sai. Đây cũng là lý do đề tài ưu tiên tiến hóa bộ nhớ/tài liệu thay vì tự động sinh thao tác cấu hình mới trên thiết bị mạng.

5. KẾT LUẬN

Đề tài đã xây dựng được một prototype Self-Evolving AI Agents cho xử lý sự cố mạng viễn thông trên nền tảng NIKA. Hệ thống tích hợp môi trường Kathará, các MCP server chẩn đoán, workflow ReAct/agent và hai cơ chế tự tiến hóa gồm Procedural Memory và Tool Refinement. Kết quả thực nghiệm trên hai lần chạy 125 case chung cho thấy hướng tiếp cận tự tiến hóa giúp tăng incident success rate từ 20.8% lên 28.0%, tăng RCA F1 từ 0.304 lên 0.387, đồng thời giảm số bước và số lần gọi công cụ trung bình.

Tính mới của đề tài nằm ở việc không xem agent như một thành phần tĩnh, mà thiết kế vòng lặp học sau mỗi episode. Tính sáng tạo thể hiện ở sự kết hợp giữa bộ nhớ thủ tục có điều kiện kích hoạt, truy xuất theo thuộc tính miền mạng và tinh chỉnh tài liệu công cụ dựa trên trace thực tế. Tính hiệu quả được minh chứng bằng cải thiện định lượng ở các metric chẩn đoán quan trọng, đặc biệt là RCA và tỉ lệ giải quyết incident hoàn chỉnh.

Về đóng góp cá nhân, sinh viên đã trực tiếp nghiên cứu các hướng self-evolving agent, thiết kế kiến trúc tích hợp với NIKA, hiện thực hai mô-đun học kinh nghiệm, xây dựng pipeline benchmark và phân tích kết

quả. Điểm quan trọng của sản phẩm là các thành phần tiến hóa đều có trạng thái rõ ràng, có thể kiểm tra lại và có thể tắt/bật để đánh giá ablation trong các thí nghiệm tiếp theo.

Trong thực tế, hệ thống có thể được phát triển thành trợ lý cho kỹ sư vận hành mạng: tự thu thập bằng chứng, đề xuất nguyên nhân, giải thích quá trình suy luận và học từ phản hồi chuyên gia. Các hướng phát triển tiếp theo gồm tăng khả năng tổng quát hóa sang topology/giao thức mới, bổ sung cơ chế human-in-the-loop để phê duyệt kỹ năng trước khi lưu lâu dài, tối ưu token khi nạp bộ nhớ và mở rộng benchmark sang multi-fault incidents có nhiều nguyên nhân đồng thời. Một hướng quan trọng khác là xây dựng dashboard giám sát quá trình tiến hóa, cho phép xem kỹ năng nào được kích hoạt, tài liệu công cụ nào đã hội tụ và episode nào đóng góp vào thay đổi bộ nhớ.

TÀI LIỆU THAM KHẢO

- [1] Anthropic. Model context protocol. <https://modelcontextprotocol.io>, 2024. 2
- [2] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, and David Walker. P4: Programming protocol-independent packet processors. In *ACM SIGCOMM Computer Communication Review*, pages 87–95, 2014. 2
- [3] Harrison Chase et al. Langgraph: Building stateful, multi-actor applications with llms. *LangChain Documentation*, 2024. 2
- [4] FRRouting Project. Frrouting: A free range routing protocol suite. *GitHub Repository*, 2021. 2
- [5] Peng Huang et al. A survey on self-evolving agents. *arXiv preprint arXiv:2407.04543*, 2024. 1
- [6] Yingying Liu et al. A survey on automated network root cause analysis. *arXiv preprint*, 2023. 1
- [7] Stefano Marrone, Luca Nicoletti, and Claudio Pizzuti. Kathará: a container-based framework for implementing network function virtualization and software-defined networking. In *IEEE International Conference on Computer Communication and Networks*, pages 1–9, 2019. 2
- [8] Stefano Martiradonna et al. Nika: Network intelligence for knowledge-driven agents – a benchmark for ai-based network troubleshooting. *arXiv preprint*, 2025. 1
- [9] Chen Qian et al. Scaling llm test-time compute for long-context reasoning via procedural memory. *arXiv preprint*, 2024. 2
- [10] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. 2
- [11] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023. 2
- [12] Junjie Xu et al. Draft: Difficulty-responsive adaptive fine-tuning for tool use in large language models. *arXiv preprint*, 2024. 3
- [13] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. 2
- [14] Shuai Zhou et al. Llm-based network fault diagnosis: A survey. *arXiv preprint*, 2024. 1